MetroInsights Phase 5 - Technology Study Roadmap

Repository basis: `dmrc_footfall_predictor`, revision `4405d167fc323f33be76e2c7f6614fd21cc574a8`, audited 2026-10-14. This sequence teaches technologies that appear in the repository and prepares you to explain their trade-offs. It distinguishes implementation from future production needs. MetroInsights currently uses generated synthetic passenger scenarios; its service parameters are configurable assumptions and its route durations are estimates. Studying the technologies does not convert those inputs into real-world validation.

How to use the roadmap

Follow the modules in order. For each one: read the concept, open the listed project files, perform the exercise without copying the existing output, then answer the interview questions aloud. Advance when you can explain the mastery checklist without relying on memorized prose.

Priority labels

• **Essential:** Needed to explain the current code confidently.

• **Advanced:** Helps defend design choices and diagnose deeper issues.

• **Optional extension:** Useful if evolving the prototype toward a larger product, but not implemented here.

Suggested sequence

Study block	Modules	Outcome
Block 1 - language and data	1-2	Read Python modules and trace tabular data
Block 2 - web request lifecycle	3	Explain startup, HTTP, templates and browser behavior
Block 3 - ML reasoning	4-5	Explain generated data, features, training and evaluation limits
Block 4 - domain calculations	6-7	Defend scheduling math and graph paths
Block 5 - reliability	8	Discuss tests, security, deployment and production gaps

One block can be split across multiple sessions. The goal is demonstrated understanding, not finishing on a fixed calendar date.

Module 1 - Python and program execution

Priority: Essential. **MetroInsights touchpoints:** `run.py`, `app/__init__.py`, `app/routes.py`, `app/forecasting.py`, `pipeline/*.py`.

Learning objective

Trace a Python program from import to function call and explain how values, containers, branches and exceptions move through it.

Prerequisites

Variables, arithmetic, strings and basic `if` statements. If those are unfamiliar, practice them before reading Flask or model code.

Core concepts and mental models

• **Import executes module setup:** importing a module is like opening its toolbox and running its top-level setup instructions. In this project, that includes file reads and model loads.

• **Functions are contracts:** inputs enter through parameters; outputs leave through `return`; side effects include reading files, changing global state or constructing responses.

• **Containers:** lists preserve order, dictionaries map keys to values, tuples can identify compound objects such as `(station, line)`, and sets hold unique names.

• **Control flow:** an `if` selects a path; a loop repeats for each value; an early `return` exits immediately; `try/except/finally` separates recovery and cleanup.

• **Comprehensions and lambdas:** compact ways to build collections or small callbacks. Expand them into ordinary loops when learning.

• **Type and value are distinct:** `None`, `0`, `False`, empty string and missing dictionary key have different meanings.

Technical explanation

Python loads a module once per process and stores its module object in `sys.modules`. The first import evaluates top-level statements in order. A function body is not run until called, but decorators are evaluated when the `def` statement runs. In Flask, `@app.route(...)` registers a function during route-module import. Global module values then remain shared inside that worker process.

Inferred annotations such as `hour: int` are documentation and tooling hints; they do not automatically validate runtime input. `dict.get(key, default)` supplies a fallback only when a key is absent, not necessarily when the value is explicitly `None`.

MetroInsights connection and practical example

Read `app/routes.py:235-398`. The function parses request fields, computes service state, conditionally calls predictors, builds recommendation data, and returns JSON. Track `service_open` like a switch: open means selected-time forecasts are computed; closed means those two demand values are assigned zero. The full-day profile call still occurs. Then inspect `app/forecasting.py:56-116` to see a returned integer prediction.

Interview questions

1. What happens at import time in `app/forecasting.py`? 2. What is the difference between `return None`, `return 0`, and no matching dictionary key? 3. Why does route registration happen when `app.routes` is imported?

Follow-ups: Which of those behaviors can prevent the process starting? How would you validate types at a request boundary?

Common mistakes: Assuming annotations enforce types; assuming import is declaration-only; overlooking module globals; confusing local assignment with mutation of a shared object.

Exercises

1. Draw the call chain from `run.py` through app initialization to `/api/predict`. 2. On paper, trace how `station`, `date`, `time`, and `line` values move into the API response. 3. Rewrite a nested comprehension from `travel_graph.py` as a regular loop, preserving its output.

Mastery checklist

• [ ] Explain the difference between module initialization and function execution.

• [ ] Trace one `if/elif/else` branch and one loop in source order.

• [ ] Identify a function's inputs, outputs, globals and side effects.

• [ ] Explain why missing, empty and zero values should not be treated as interchangeable.

Advanced and optional topics

Advanced: Python object identity, mutable defaults, exception hierarchy, decorators and import cycles. **Optional extension:** static type checking and explicit dependency injection to reduce import-time coupling.

Module 2 - Tabular data, NumPy, and reproducible pipelines

Priority: Essential. **Project files:** `pipeline/01_audit_and_clean_data.py`, `pipeline/02_generate_calibrated_dataset.py`, `pipeline/03_train_and_evaluate_forecaster.py`, `data/`.

Learning objective

Follow a row from raw station metadata through cleaning, synthetic generation and training input, and identify where each value originated.

Prerequisites

Module 1; CSV rows/columns; basic mean and probability.

Core concepts and mental models

• **DataFrame as a labeled table:** rows are records; columns name fields; indexes are lookup labels, not always row numbers.

• **Schema as a contract:** station name, date, hour and target must have understood types and meanings.

• **Cleaning versus verification:** formatting/correcting a value does not prove it is externally true.

• **Random seed as a repeatable starting point:** it can reproduce draws for the same software/data/order; it does not make generated values observations.

• **Pipeline stages:** each stage transforms an input into an artifact consumed by the next stage.

• **Synthetic vs observed:** generated samples encode the assumptions in the generator.

Technical explanation

Pandas reads CSV text into typed columns. Operations such as `groupby`, `merge`, `map`, `fillna`, and `to_datetime` create transformed columns/tables. NumPy provides vector operations and random sampling. A Poisson draw with expectation `lambda` produces a nonnegative integer whose average tends toward `lambda` over repeated draws; an individual sample varies.

MetroInsights pipeline 02 creates 228 x 728 x 19 = 3,153,696 station/date/hour records. `seed=42` supports repeatability but only under stable generator/version/input order. Pipeline 01 includes hand-authored values and corrections; the word "cleaned" means transformed into a usable format, not independently certified.

MetroInsights connection and practical example

Inspect pipeline 02's date/hour loops, station weights and output columns. For one station/date/hour, distinguish the deterministic expected value assembled from line/station/day/time factors from the random Poisson sample ultimately written. Then inspect `.gitignore`: the generated large CSVs are ignored, but the pipeline still writes them locally. Serving loads saved model artifacts; training consumes the generated CSV.

Interview questions

1. What does the random seed guarantee, and what does it not guarantee? 2. Why is 3.15 million rows not equivalent to 3.15 million real observations? 3. How would you validate a station CSV before allowing it into training?

Follow-ups: What should happen if normalized station names collide? How can chunked processing help with large CSV memory use?

Common mistakes: Calling imputed/manual values verified; claiming synthetic data improves real-world accuracy; assuming `.gitignore` deletes data; ignoring units or duplicate rows.

Exercises

1. Calculate the expected row count from the pipeline dimensions before running it. 2. Write a small schema checklist for the raw station CSV and operations CSV. 3. Explain how you would prove that two runs with seed 42 are repeatable and record the conditions that could break repeatability.

Mastery checklist

• [ ] Name the producer and consumer of each major CSV/JSON artifact.

• [ ] Separate source observations, manual assumptions and generated values.

• [ ] Explain rows x dates x hours arithmetic.

• [ ] Describe reproducibility and its limits.

Advanced and optional topics

Advanced: vectorization, chunked CSV reading, memory profiles, schema evolution and data lineage. **Optional extension:** a versioned data catalog, object storage, immutable raw snapshots and automated validation reports.

Module 3 - HTTP, Flask, Jinja, and browser JavaScript

Priority: Essential. **Project files:** `app/__init__.py`, `app/routes.py`, `Procfile`, `app/templates/`, `app/static/js/`.

Learning objective

Explain how a selection in the browser becomes a Python request and how a JSON response becomes the displayed dashboard.

Prerequisites

Modules 1-2; basic HTML/JavaScript objects and functions.

Core concepts and mental models

• **HTTP request/response:** a request carries method, path, headers and body; a response carries status, headers and a body.

• **JSON contract:** client and server agree on field names and types.

• **WSGI app:** Gunicorn loads a Python module/object and forwards HTTP requests to Flask.

• **Template vs API:** Jinja produces initial HTML; JSON endpoints return data for browser code.

• **Promise/`async`/`await`:** waiting for network I/O does not freeze the browser's entire event loop.

• **DOM:** JavaScript can find nodes by IDs and update their text/attributes.

• **Stale response protection:** request sequence numbers prevent late old answers overwriting newer state.

Technical explanation

`Procfile` starts `gunicorn run:app`. Importing `run` imports the Flask package; route decorators register handlers during import. The browser posts JSON to `/api/predict`; Flask parses the body, orchestrates Python modules and returns `jsonify(...)`. JavaScript awaits the response and maps named fields to DOM IDs. A line filter changes which schedule headline/curve is displayed. HTTP error codes should be checked separately from whether a body parses as JSON; current `fetchPrediction` does not inspect `res.ok`.

MetroInsights connection and practical example

Trace `dashboard.js:154-201` to `routes.py:235-398`, then `dashboard.js:203-313`. Request body keys are `station`, `date`, `time`, `line`. Response sections include `datetime`, `calendar_context`, `footfall`, `scheduling`, `profile_24h`, and `travel_times`. The stale ID blocks outdated rendering but does not abort network requests.

Interview questions

1. What is the difference between an HTML route and `/api/predict`? 2. Why can two asynchronous responses arrive out of order? 3. What is the role of `Content-Type: application/json`?

Follow-ups: What should the client do with a 400 response? How would you prevent repeated slider requests?

Common mistakes: Calling request sequencing cancellation; assuming the form's action is the API path; treating Jinja autoescaping as protection for client-side `innerHTML`.

Exercises

1. Draw request and response JSON shapes from the route and frontend. 2. Add a written validation contract: allowed stations, dates, times and line values. 3. Trace a malformed date to HTTP status and visible browser message.

Mastery checklist

• [ ] Explain browser -> HTTP -> Flask -> module calls -> JSON -> DOM.

• [ ] Identify which side owns each response field.

• [ ] Explain async fetch and stale-response protection.

• [ ] Distinguish server template escaping from safe DOM rendering.

Advanced and optional topics

Advanced: WSGI worker/process models, CORS, caching, content negotiation and API versioning. **Optional extension:** typed request schemas, OpenAPI, CSRF/auth controls if user accounts are introduced, and an `AbortController`/debounce strategy.

Module 4 - Statistics, feature engineering, and supervised learning

Priority: Essential. **Project files:** `pipeline/03_train_and_evaluate_forecaster.py`, `app/forecasting.py`, `models/`.

Learning objective

Explain what the model learns, how one inference row is built, how metrics are interpreted and why synthetic evaluation has a limited claim.

Prerequisites

Modules 1-3; average, variance, train/test split, regression and basic probability.

Core concepts and mental models

• **Supervised regression:** learn a mapping from feature vector X to numeric target y.

• **Feature engineering:** create useful input representations from raw time/station context.

• **Cyclical features:** sine/cosine encode that the end of a cycle is near its beginning.

• **Baseline:** a simple predictor provides a reference point a complex model should beat.

• **Chronological holdout:** evaluate on a later time range to mimic future prediction.

• **Leakage:** information from the target/future accidentally enters a feature or preprocessing fit.

• **MAE/RMSE/R-squared/MAPE:** complementary measures; each has assumptions and interpretation.

Technical explanation

The model receives one row with 23 columns: time/calendar fields, cyclical encodings, categorical indexes, station and typology metadata, and historical means. Metadata stores `feature_cols`; runtime builds the row in that exact order. `HistGradientBoostingRegressor` learns nonlinear splits/updates through additive trees. Ridge regression provides a regularized linear baseline. Metrics are calculated on the generated dataset's chronological holdout. MAPE excludes zero targets because percentage division by zero is undefined.

The right mental model for evaluation: "How well did the model recover the generator's later scenarios?" It is not "How well will the model predict actual passengers?"

MetroInsights connection and practical example

Take a selected station/date/hour and trace how `predict_hourly_footfall` derives weekday/month, holiday/workday category, sine/cosine values, station index, historical values and typology index. It selects the feature columns, calls `.predict`, converts to float, rounds and clamps below zero. Minute interpolation happens after hourly predictions; it is not trained minute-level inference.

Interview questions

1. Why preserve model feature order in metadata? 2. Why does a chronological split matter for forecasting? 3. Why are a baseline and error analysis needed if R-squared is high?

Follow-ups: How would you detect leakage in historical means? How would you validate on real data?

Common mistakes: Treating feature importance as causation; reporting synthetic test metrics as real accuracy; believing a random split is always suitable for time series; ignoring MAPE's zero behavior.

Exercises

1. Hand-compute hour sine/cosine features for 00:00 and 23:00 and explain their proximity. 2. Sketch the difference between MAE and RMSE if there is one large miss. 3. Propose a small leakage test using a three-date fixture.

Mastery checklist

• [ ] Describe all broad feature families used by inference.

• [ ] Explain training vs inference feature schema.

• [ ] Interpret each saved metric accurately.

• [ ] State exactly what synthetic holdout results can and cannot support.

Advanced and optional topics

Advanced: calibration, residual stratification, uncertainty intervals, concept drift, grouped/time-based cross-validation and target transforms. **Optional extension:** compare gradient boosting to seasonal naive forecasts and probabilistic models on verified observed data.

Module 5 - Operating assumptions and scheduling mathematics

Priority: Essential. **Project files:** `app/operating_hours.py`, `app/scheduling_optimizer.py`, `data/cleaned/metro_line_operations.csv`.

Learning objective

Explain the units in the service recommendation, reproduce its arithmetic and identify feasibility gaps.

Prerequisites

Modules 1-4; ratios, floor/ceil, units and percentages.

Core concepts and mental models

• **Headway:** time gap between trains. Smaller headway means higher frequency.

• **Frequency conversion:** trains/hour = 60 / headway in minutes.

• **Fleet cycle:** trains needed is approximately round-trip cycle time divided by headway.

• **Capacity flow vs occupancy:** passengers/hour that can board is a flow rate; passengers onboard is a stock.

• **Constraint:** a candidate recommendation must satisfy every fleet, signaling and policy bound after rounding.

• **Boundary semantics:** `start <= t < end` includes start and excludes end.

Technical explanation

The scheduling helper derives frequency bounds from fleet/turnaround and minimum signaling headway. It calculates per-train boarding clearance from comfort capacity and an assumed through-load, compares station demand with baseline hourly clearance, chooses an action, clamps a target frequency, converts to headway and rounds to the nearest standard value. That final rounding can raise frequency above the earlier clamp; the Yellow Line example in Phase 2 explains the tie. The current utilization is `station line demand / modeled hourly clearance * 100`, not onboard occupancy.

The operating-hours utility uses a 05:00 inclusive, 23:30 exclusive interval. This is a modeled gate, not a verified live operations calendar.

MetroInsights connection and practical example

Use the Yellow Line assumptions: comfort capacity 1,920; assumed peak through-load 65%, giving `1920 * (1 - .65) = 672` clearance riders/train. At 26.1 trains/hour, modeled clearance is `672 * 26.1 = 17,539.2`, then integer conversion gives 17,539. If demand were 5,000 riders/hour, utilization would be about `5000 / 17539 * 100 = 28.5%`. This is a model calculation, not measured occupancy.

Interview questions

1. What is headway and how does it relate to frequency? 2. What denominator does this project's capacity utilization use? 3. Why is the post-rounding fleet invariant important?

Follow-ups: What data would be needed to estimate onboard load? What does the floor operation do to a capacity bound?

Common mistakes: Calling boarding clearance onboard load; assuming operations CSV values are official; checking constraints before rather than after rounding; treating exactly 23:30 as open.

Exercises

1. Convert 5-minute headway to trains per hour and expected random-arrival wait. 2. Compute clearance for comfort capacity 1,600 and 40% through-load. 3. Write pseudocode that chooses a practical headway while preserving fleet and signaling constraints after rounding.

Mastery checklist

• [ ] Keep units consistent across minutes, trains/hour and riders/hour.

• [ ] Explain the difference between nominal, comfort, crush and boarding-clearance values.

• [ ] Reproduce one schedule calculation by hand.

• [ ] State the practical-rounding risk and a postcondition test.

Advanced and optional topics

Advanced: queueing theory, passenger arrival distributions, dwell-time/capacity feedback, integer programming and timetable coordination. **Optional extension:** train-level state simulation with boarding/alighting and origin-destination data.

Module 6 - Graphs, geographic distance, and shortest paths

Priority: Essential. **Project files:** `app/network_topology.py`, `app/travel_graph.py`.

Learning objective

Read the project's line-expanded graph and explain why Dijkstra returns an estimated shortest network path.

Prerequisites

Modules 1-2; dictionaries, tuples and basic geometry.

Core concepts and mental models

• **Graph:** vertices represent states; edges represent possible transitions; edge weight is cost/time.

• **Adjacency map:** for each node, list its neighbors and edge weights.

• **Line-expanded node:** `(station, line)` retains platform/corridor state.

• **Transfer edge:** a same-station edge between different line nodes charges transfer time.

• **Relaxation:** replace a known distance only when a newly found path is shorter.

• **Min-heap:** efficiently select the next lowest tentative travel time.

• **Haversine:** spherical great-circle distance between coordinate points, not track distance.

Technical explanation

The topology file lists stations in line order. The builder maps topology labels to canonical CSV station names, creates edges only between consecutive stops, estimates or falls back to travel costs, and connects listed interchanges. Dijkstra uses a priority queue and nonnegative edge weights; after the search it collapses line-nodes to the minimum time per station. The current origin starts from each of its line nodes at zero.

MetroInsights connection and practical example

On a toy path A-X to B-X = 3 min, B-X transfer X-to-Y = 5 min, B-Y to C-Y = 2 min, the result to C is 10 min. If direct A-X to C-X were 12 min, Dijkstra chooses 10. Actual MetroInsights edges may use coordinate-derived straight-line cost or a 2.5-minute fallback; all transfers use a generic 5-minute penalty.

Interview questions

1. Why is the node a `(station, line)` tuple? 2. Why must Dijkstra edge costs be nonnegative? 3. Why aren't the resulting times timetable predictions?

Follow-ups: What is the complexity? What happens when a topology station name fails to map?

Common mistakes: Assuming geography creates an edge; calling haversine rail distance; forgetting transfer costs; using Dijkstra with negative weights.

Exercises

1. Run Dijkstra by hand on a four-node graph and show each heap pop/relaxation. 2. Add a transfer edge to a drawing and compare its cost to remaining on one line. 3. Identify a station in topology/CSV that would need an alias and explain why mapping is important.

Mastery checklist

• [ ] Explain vertices, adjacency, weights and transfer state.

• [ ] Trace one relaxation and one stale heap entry.

• [ ] State nonnegative-weight requirement and complexity.

• [ ] Clearly label topology travel durations as estimates.

Advanced and optional topics

Advanced: A*, bidirectional Dijkstra, time-dependent edge weights and route reconstruction. **Optional extension:** GTFS schedules, transfer walking graphs and real-time arrival updates after sourcing/validating appropriate data.

Module 7 - Software reliability, security, testing, and deployment

Priority: Essential concepts; some improvements are not implemented. **Project files:** `Procfile`, `requirements.txt`, `.gitignore`, `app/routes.py`, `app/static/js/dashboard.js`, model artifacts.

Learning objective

Describe what has to be present for serving, what the API validates, and how to verify behavior safely.

Prerequisites

Modules 1-6; basic command line and Git concepts.

Core concepts and mental models

• **Build vs runtime:** installing dependencies and launching the WSGI app are separate stages.

• **Inference artifact vs training dataset:** the web worker loads model files; offline training reads the large generated CSV.

• **Validation boundary:** reject malformed or unsupported inputs before domain logic.

• **Test pyramid:** unit tests for pure logic, integration tests for route/module contracts, browser tests for UI behavior.

• **Safe output:** use text nodes for untrusted text; validate dynamic style values.

• **Observability:** log enough to diagnose errors without exposing sensitive data.

• **Reproducibility:** pin runtime/dependencies and version compatible model/metadata artifacts.

Technical explanation

Render uses `web: gunicorn run:app`. Required files are read during imports, so missing or incompatible artifacts can prevent worker startup. `.gitignore` excludes large generated training CSVs but does not delete them or add them to Render. Serving works from committed model/metadata and compact runtime data; retraining requires the generated dataset or a regeneration step. `requirements.txt` mixes pins and lower bounds, and no lockfile or automated tests are tracked.

Browser output uses `innerHTML` for some dynamic row values; this should be replaced with DOM nodes and `textContent` where appropriate. Joblib artifacts should be loaded only from trusted project-controlled sources.

MetroInsights connection and practical example

Test `is_service_operating` at 04:59, 05:00, 23:29 and 23:30. Test headway outputs against a post-rounding fleet/signaling invariant. Use Flask's test client to send valid/invalid JSON and verify status/schema. Mock a browser response to check KPI DOM updates and stale-response handling. These tests would be additions; no suite currently exists.

Interview questions

1. Which large file is required for serving versus retraining? 2. Why can module-level model loading fail a Gunicorn deployment? 3. What are the security concerns with `innerHTML` and joblib?

Follow-ups: How do you prove artifact compatibility? What is the first test you would write?

Common mistakes: Assuming Git ignore means file is deployed; claiming tests pass when none exist; relying on manual UI checks only; treating saved model files as version-independent.

Exercises

1. Draft test cases for date parse failure, unknown station, closed hour and unknown line filter. 2. Design a minimal model artifact manifest with model version, feature schema, training revision and dependency versions. 3. Draw the Render build/startup path and label which files are loaded at each stage.

Mastery checklist

• [ ] Explain Render's start command and module import path.

• [ ] Distinguish serving artifacts from retraining inputs.

• [ ] Name at least four high-value boundary/integration tests.

• [ ] Identify current XSS/serialization risks accurately.

• [ ] Separate present behavior from recommended improvements.

Advanced and optional topics

Advanced: structured logging, health/readiness endpoints, worker memory/concurrency, artifact signatures, CI pipelines and dependency lockfiles. **Optional extension:** data/model monitoring, drift alerts, access control and durable scenario history.

Module 8 - Product reasoning and model/system limitations

Priority: Essential for interviews and responsible project claims. **Project files:** `DATA_AUDIT.md`, `app/routes.py`, `app/scheduling_optimizer.py`, `app/travel_graph.py`.

Learning objective

Defend what the project can demonstrate, articulate what it cannot infer, and prioritize a technically justified next step.

Prerequisites

Modules 1-7.

Core concepts and mental models

• **Prediction vs decision:** demand estimates and schedule recommendations are distinct stages with separate uncertainty.

• **Flow vs stock:** riders/hour is a flow; people onboard is a stock at an instant.

• **Simulation vs evidence:** a scenario can illustrate a mechanism without proving the mechanism matches reality.

• **Data provenance:** source, retrieval date, license, transformations and quality checks should accompany each dataset.

• **Validation target:** define the real outcome and evaluation period before selecting a more complex model.

Technical explanation

The dashboard forecast is based on synthetic training labels. The scheduler converts demand into a boarding-clearance scenario using assumed through-load. The graph converts topology and approximate segment costs into shortest-time estimates. None is a live source. Improving claim quality requires replacing assumptions with sourced passenger, operating and network data, then validating each subsystem against an appropriate observed target.

MetroInsights connection and practical example

When the dashboard reports capacity utilization, explain its numerator and denominator. When it reports a destination time, explain the graph weights. When a metric is high, identify the synthetic evaluation set. This is how you defend the project without overstating it.

Interview questions

1. What is the highest-priority limitation to fix? 2. Why is adding a more complex model not necessarily the next best step? 3. How would you explain this project to a nontechnical planner?

Follow-ups: What exact data would you request? How would you preserve provenance and privacy?

Common mistakes: Calling a prototype production-ready; making claims of reduced crowding; treating UI detail as data validation; prioritizing model sophistication over target validity.

Exercises

1. Write a two-sentence public description that labels forecasts and recommendations as estimates. 2. Create a data acceptance checklist: source, license, timestamps, station IDs, missingness, counts, update cadence and validation. 3. Draw an onboard passenger state update using prior load, boardings and alightings; list the missing project data.

Mastery checklist

• [ ] Explain three separate estimation layers: model, scheduler and graph.

• [ ] Identify the current evidence limit of each layer.

• [ ] State a useful next step and why it has higher value than an algorithm rewrite.

• [ ] Discuss project work truthfully without inventing personal impact.

Advanced and optional topics

Advanced: uncertainty propagation from forecast into capacity planning, decision analysis and human oversight. **Optional extension:** privacy-preserving AFC analytics, fairness checks across stations, and controlled operational pilots after approval.

Capstone sequence

Complete these in order after the modules:

1. Trace one request from the browser payload to every response section and identify each module boundary. 2. Recalculate one station/hour schedule by hand and check all constraints after rounding. 3. Explain one Dijkstra route by listing line-nodes, track edges, transfer cost and final minimum. 4. Design a tiny observed-data evaluation without using any future target information. 5. Give a two-minute explanation that states one strength, two limitations and one next step.

Final readiness rubric

• **Ready to explain:** You can trace values and reproduce arithmetic with the source open.

• **Ready for follow-ups:** You can explain assumptions, edge cases, alternatives and failure modes.

• **Ready to defend claims:** You distinguish generated simulation results from measured operational outcomes.

• **Ready to improve:** You can propose tests and a data-backed next step before recommending more complexity.